



AI AGENT WORKSHOP TOKYO 2026 — 第一部

Agentic RLの基礎からハンズオン

OpenPipe ART × W&B Training × Weave で、メール検索エージェントを強化学習する

<https://github.com/nejumi/art-e-nano>

ワークショップの流れ

1・イントロと強化学習の基礎	座学
2・Agentic RLと報酬設計	座学
3・ハンズオン① ART-Eを動かす	実習:学習を開始
4・GRPO / GSPO	座学(学習を待つ間)
5・フレームワークと OpenPipe ART	座学(学習を待つ間)
6・ハンズオン② 結果を見て改善	実習:もう一度、学習を開始
7・トラブルシューティング	座学(学習を待つ間)
8・まとめと次の一歩	座学

学習の実行時間を活かし、**走らせる → 待つ間に学ぶ → 結果を見て直す**、と座学と実習を往復します

受講後にできるようになること

4つの動詞で持ち帰る

Define

Agentic RLの問題を、観測・行動・報酬・軌跡として定義できる

Train

ARTでScenario → rollout → TrajectoryGroup → trainの流れを読める

Observe

W&Bで報酬・正答率・turn数・IDK率を追い、Weaveで軌跡を見られる

Debug

報酬が上がらない・崩壊する・報酬だけ上がるケースを切り分けられる

イントロと強化学習の基礎

解き方は教えず、結果の正しさだけを手がかりに学習する

模範解答の手順をなぞるのではなく、結果が正しいか(検証可能な報酬)を信号に方策を磨いていく



AlphaGo

自己対戦をくり返し、勝敗という報酬から打ち手を改善。世界トップ棋士を破った



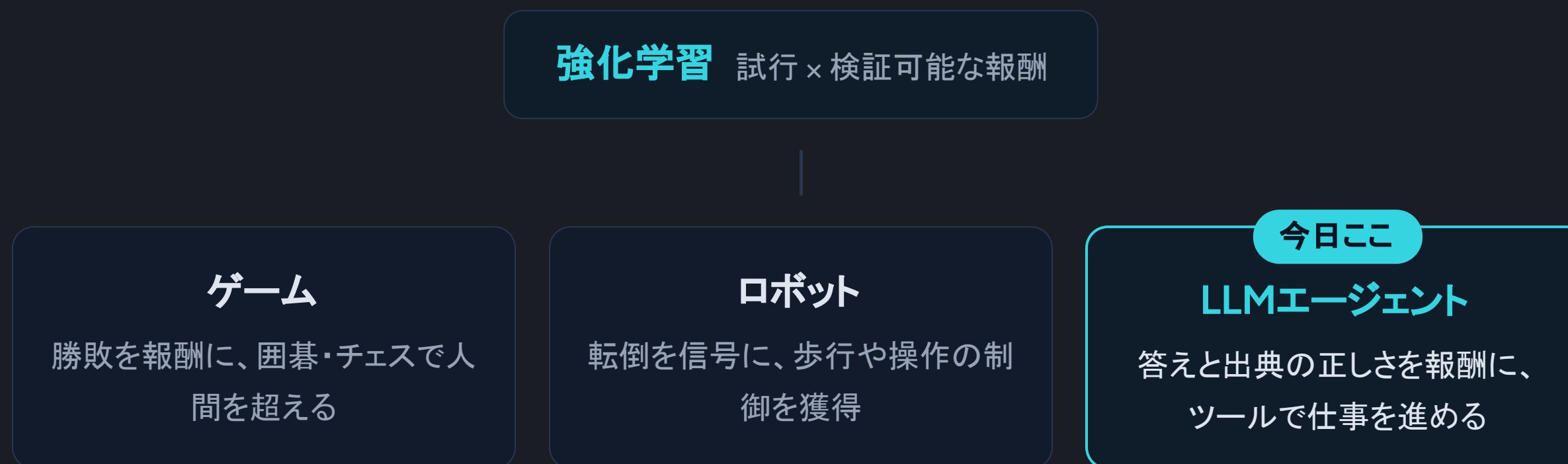
ヒューマノイド

転倒を失敗信号として、歩行やバランスの制御を自分で獲得していく

手本をなぞるSFTと異なり、**検証できる「結果の正しさ」から方策(policy)を最適化する**。今日のART-Eも同じ枠組みで学習する

強化学習は、領域を問わない汎用の枠組み

行動を試し、検証可能な結果(報酬)から方策を更新する——この構造はゲームも制御もLLMも共通



今日はこの枠組みを「**LLMエージェント**」に適用し、メール検索エージェントを題材に体験します

大きなモデルに頼るだけでは、実務の Agent は安定しない

- 01 汎用モデルは強いが、企業ごとの業務手順・ツール・ポリシーには最適化されていない
 - 02 Prompt だけでは、失敗後の立て直し・ツールの打ち切り・引用の正確さを安定化しづらい
 - 03 SFT だけでは良い例の模倣に寄りやすく、探索から新しい成功パターンを見つけにくい
- Agentic RL は「成功した仕事の進め方」をモデルに増やすための方法

強化学習の基本用語

試行 → 結果を評価 → 良い行動の確率を上げる

Policy 状況を見て次の行動を選ぶルール

Action 実際に選んだ行動。次のtoken、tool call、最終回答

Reward その行動や軌跡が良かったかを表す点数

Trajectory 観測・行動・tool結果・最終回答までの履歴

Training 報酬が高い軌跡に近づくようにpolicyを更新する

長期的な報酬を上げたい。でも、モデルを壊したくない

単純にやると起きる問題

- たまたま成功した行動を過剰に強化する
- 1回の更新で出力分布が急に変わる
- 報酬を得る抜け道を学ぶ
- 成功した理由が途中のどの行動か分からない

実務での問い

何を報酬にし、どれくらい安全に policy を動かすか？



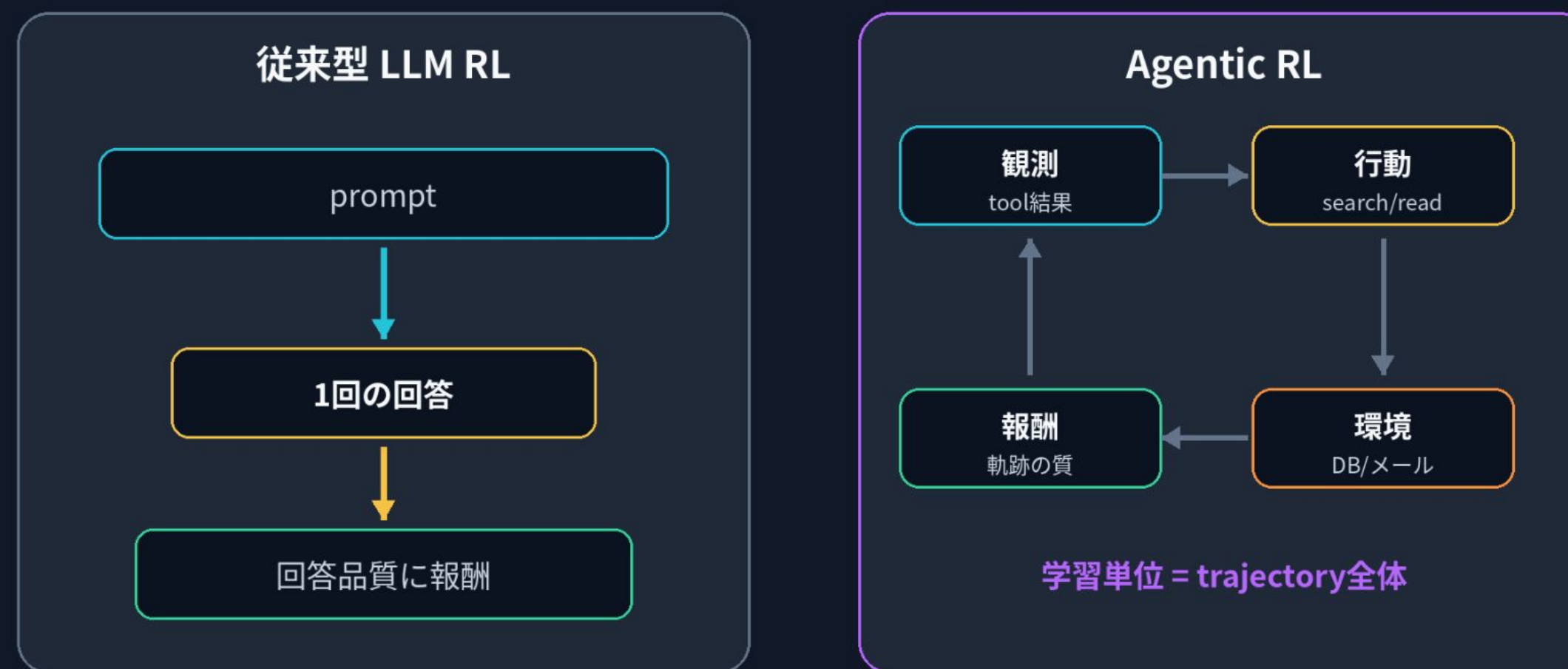
強化学習の主要手法を地図で見る

REINFORCE	報酬で直接policyを動かす	基本形
Actor-Critic	行動役と評価役を分ける	Criticで分散を下げる
TRPO	方策を変えすぎない	安全な更新の思想
PPO	clipでTRPOを実用化	RLHFの代表格
DPO	選好ペアから直接学習	単発応答の調整に強い
GRPO	Criticなしでグループ比較	今日のハンズオン主役
GSPO	sequence単位で更新	長い軌跡の安定化に効く

Agentic RLの考え方と報酬設計

従来のLLM RLとAgentic RLの違い

従来型LLM RLとAgentic RLの違い



Agentic RLは「良い文章」ではなく、「**良い仕事の進め方**」を学習する

Agentは世界を全部見ていない

部分観測 (POMDP) から、今の状況を推定して行動する

見えるもの(観測)

検索結果

toolの返り値

画面やログ

会話履歴

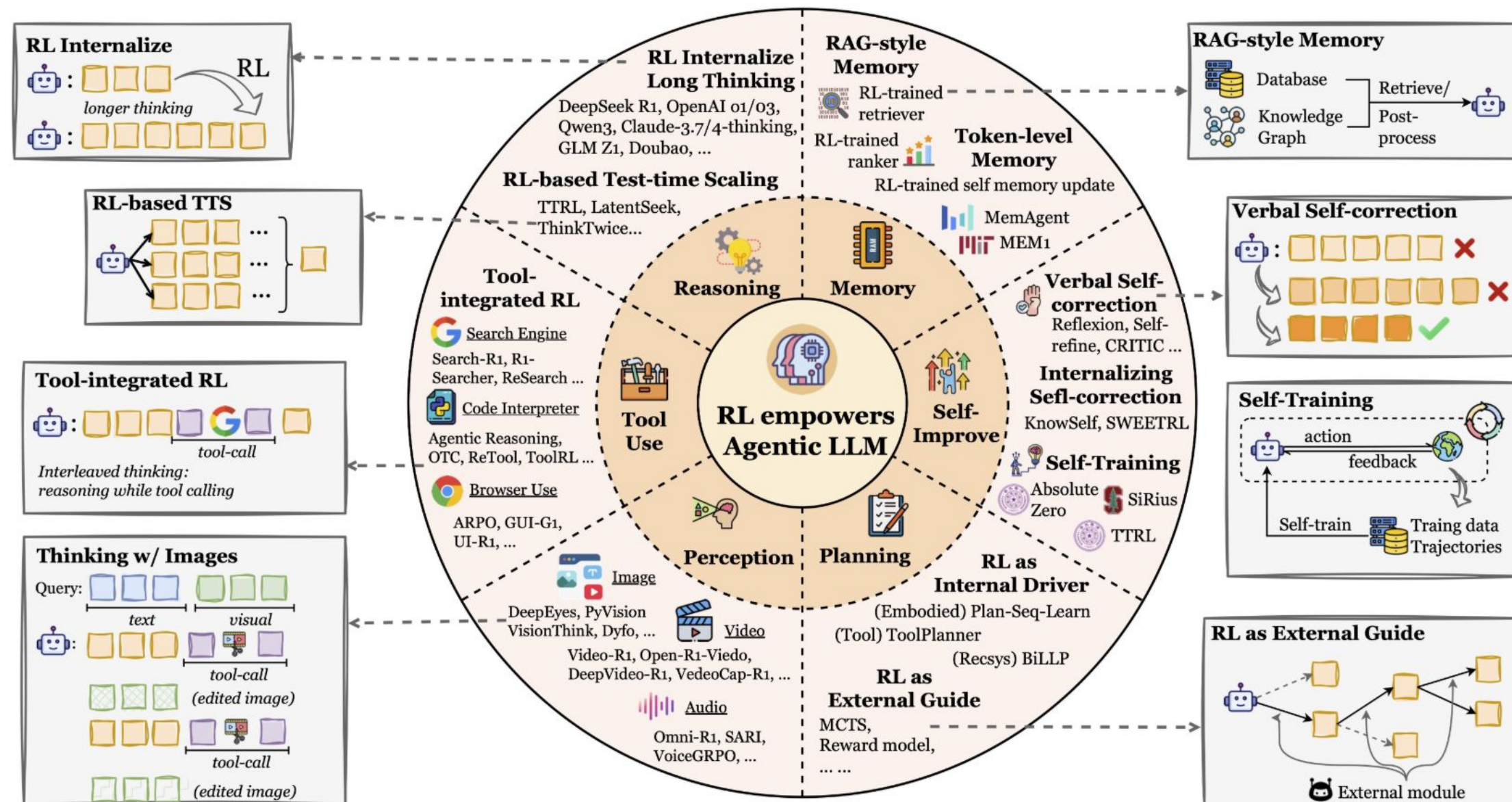
隠れているもの(実状態)

DBの全状態

メール全体

ユーザーの本当の意図

Agentic RLで学習対象になる 6つの能力



業務範囲を限定すれば、大きなモデル依存を下げられることも

性能 目的タスクに特化した探索戦略を学べる

信頼性 出典・tool手順・停止判断を改善できる

コスト 小さめのモデルにタスク固有の行動を覚えさせられる

レイテンシ 不要なtool callや長すぎる推論を減らせる

継続改善 失敗軌跡を新しい学習データにできる

Agentic RLが向くタスク、向かないタスク

向く

検索・調査・コード修正・GUI操作

成功判定や検証器が作れる

tool利用が性能ボトルネック

失敗ログが集められる

狭い業務から始められる

向きにくい

1回の自然な返答だけが欲しい

報酬が主観だけで不安定

toolがなく、単発応答で完結

環境を再現・resetできない

汎用Agentをいきなり目指す

モデルだけではない。環境込みで設計する

Agentic RL設計の5点セット

01 Environment Agentが何を見て、何を操作できるか

02 Tools 呼び出せる関数・引数・エラー・制約

03 Reward 成功・効率・安全性・品質の点数化

04 Rollout collector 軌跡をどう収集・保存・再実行するか

05 Observability W&BとWeaveで何を観測するか



まず最終成果を主軸にする

要素	例	意味
Answer correctness	正解と意味的に一致	最重要
Source correctness	正しい message_id を引用	調査Agentで重要
Fewer turns	少ないturnで正解	レイテンシのproxy
IDK handling	見つからないときの退避	hallucination抑制
Format / tool validity	JSONやtool名が正しい	学習可能な前提

報酬設計の4つの道具

検証可能な「正解／不正解」を背骨に、3つの補助で形を整える

① 主軸: 検証可能な正解／不正解

答えと出典が合っているか。機械的に判定でき、報酬の背骨になる

② 部分点: スパース性に対処

正解の途中(正しいメールを見つけた等)にも小さな信号を与える

③ ペナルティ: 望ましくない動作を抑制

誤答の断言・形式崩れ・無駄なtool呼び出しを減点する

④ LLM Judge (RULER): 品質を補強

定量化しにくい回答の妥当性を、LLMの採点で補う

補助(②③④)は、いつも**主軸①を超えない**ように設計する

LLM judge と heuristic judge

llm

OpenAI judge。意味的な正誤判定に強い

注意: APIキー・コスト・揺れ

heuristic

文字列類似。API不要で再現しやすい

注意: 表現ゆれに弱い

APIキーが無い場合も `JUDGE_MODE=heuristic` で参加可能。RULERは主報酬を置き換えず、補助加点として扱う

ハンズオン① まずはART-Eを動かしてみよう

CASE STUDY

ART・E: メールボックス検索で 質問に答えるAgent

ユーザーの質問例

「金曜日の兄のフライトは何時に到着しますか？」

- 1 質問から検索キーワードを考える
- 2 メールを検索する
- 3 関連メールを読む
- 4 答えと出典 message_id を返す
- 5 見つからなければ無理に答えない

ART・E: メール検索Agentのループ



DATASET

「エンロンのメール」が今回の題材になっている

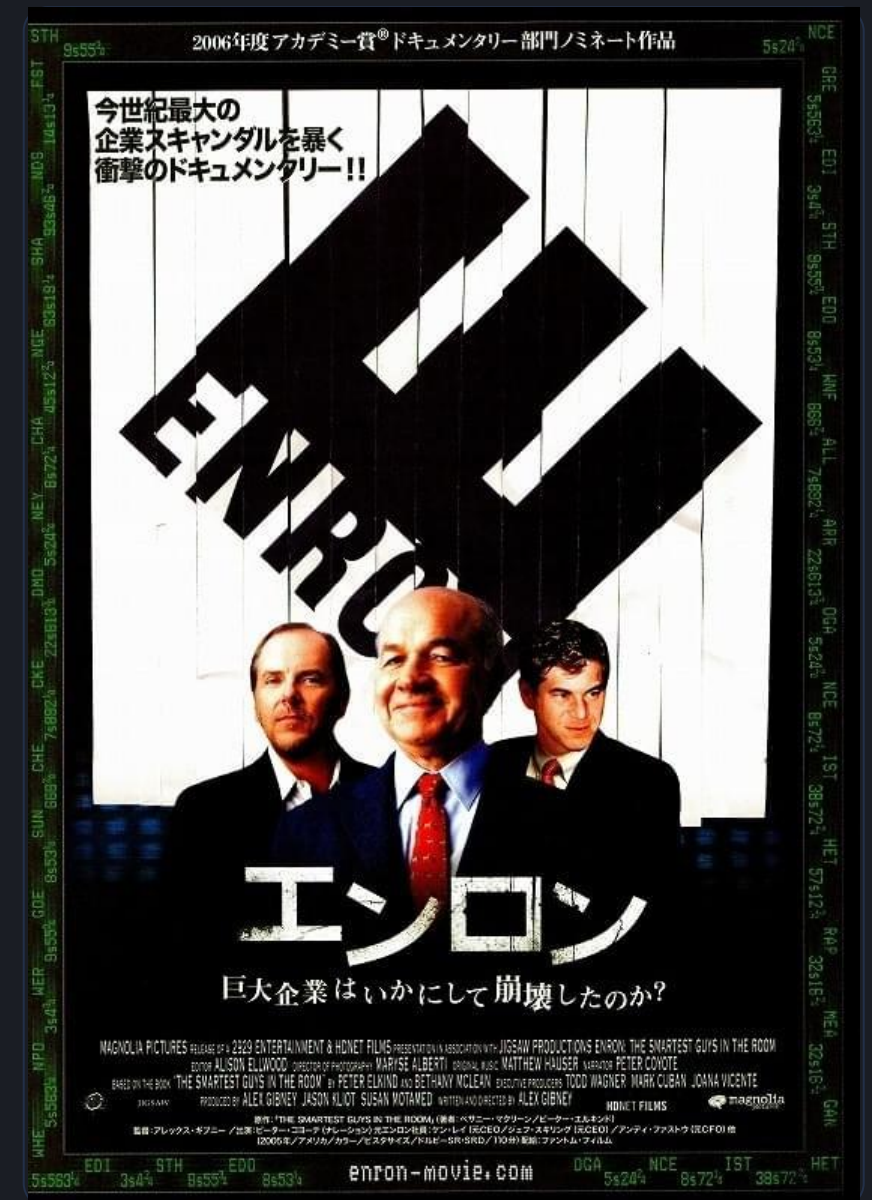
- 2001年、米エネルギー大手エンロンが粉飾会計で破綻。
米国史上有数の企業不正事件
- 訴訟の過程で、約50万通の社内メールが規制当局
(FERC)により公開された
- この実在メール群が、研究用の「Enron Email Dataset」として
広く使われている
- ART-Eはこれを受信箱に見立て、現実的な検索・QA環境を
作る

2001年 粉飾で破綻

約50万通 公開された社内メール

実在の業務 本物の受信箱

研究定番 標準データセット



ドキュメンタリー『エンロン 巨大企業はいかにして崩壊したのか?』

Enron Email Datasetに含まれるEメールの例

CEO交代を社内に案内するメール

message_id: <24926182.1075855666013.JavaMail.evans@thyme>
date: 2000-12-13 14:34:00
from: public.relations@enron.com
to: all.houston@enron.com
subject: Ken Lay and Jeff Skilling on CNNfn

原文全文:

Ken Lay and Jeff Skilling were interviewed on CNNfn to discuss the succession of Jeff to CEO of Enron. We have put the interview on IPTV for your viewing pleasure. Simply point your web browser to <http://iptv.enron.com>, click the link for special events, and then choose "Enron's Succession Plan." The interview will be available every 15 minutes through Friday, Dec. 15.

日本語訳:

Ken LayとJeff SkillingがCNNfnのインタビューを受け、JeffがEnronのCEOに就任する件について話しました。

皆さんにご覧いただけるよう、このインタビューをIPTVに掲載しました。Webブラウザで <http://iptv.enron.com> を開き、special events のリンクをクリックしてから、"Enron's Succession Plan" を選んでください。

このインタビューは、12月15日金曜日まで15分ごとに視聴できます。

メール検索 Agentを、仕事ができる方策にする

入力

ユーザーの自然言語質問
対象メールアドレス
今日の日付

環境

EnronメールをSQLite DB化
FTS5で全文検索
一度に全メールは読めない

出力

最終回答
根拠の message_id
無ければ I don't know

ART-Eの3つのtool

search_emails

キーワード・送信者・日付で最大10件検索

どんなキーワードで探すか

read_email

message_idで本文を読む

どの検索結果を読むか

return_final_answer

answerとsourcesを返す

いつ止まり、何を出典にするか

ART-Eの単純な Agent loop

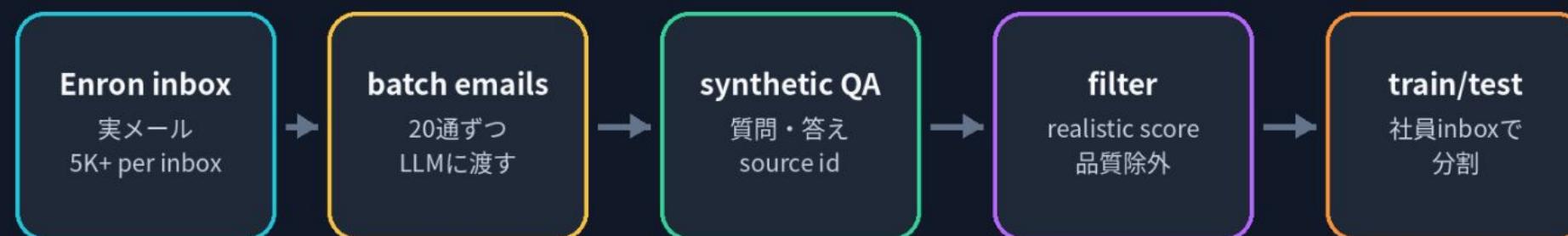


2～4を繰り返し、LLMが続きを生成し続ける

現実的なinbox + 合成QA

環境は現実的、質問は合成

ART-Eデータセット: 現実的なinbox + 合成QA



環境は現実的、質問は合成。Agentic RLでは環境のリアルさが重要。

ART・E nanoのループリック報酬

最終成果を主軸にし、部分点は補助として設計する(既定は **IDK_PENALTY=0.2**。①では逃避を観察するため 0 で開始)

ART・Eの報酬設計

最終成果を主軸にし、部分点は補助として設計する

主目的



形式とtoolが正しい
少ないturnで正解
正しいメールを読む
sourceが正しい
回答が正しい

プラス報酬

減点で抑える行動

- 誤答を断言
- IDKへ逃げすぎ
- JSON/tool不正
- turn切れ

部分点が主目的を超えないようにする



ワークショップ用の軽量化設定

Base model	OpenPipe/Qwen3-14B-Instruct	Backend	W&B Serverless RL
Max turns	6	Groups per step	12
Trajectories per group	4	Rollouts per step	48
Max steps	30	Validation	seed=42 固定96問
Learning rate	1.2e-5		

ハンズオンのリポジトリ構成

```
art-e-nano-main/  
├─ README.md  
├─ SETUP.md  
├─ .env.sample  
├─  
art_e/ |   ├─ train.py |   ├─ rollout.py |   └─  
email_search_tools.py  
|   ├─ project_types.py  
|   ├─ data/  
|   └─ evaluate/  
└─  
scripts/   └─ preflight_check.py
```

train.py

ARTの学習ループ

rollout.py

Agent loopと報酬計算

email_search_tools.py

search / read tool

project_types.py

training config

事前準備: .env を作る

```
$ cp .env.sample .env

WANDB_API_KEY=
<your-wandb-api-key> WANDB_ENTITY=<training-enabled-team-entity>
WANDB_PROJECT=art-e-nano # OpenAI judgeを使わない場合
JUDGE_MODE=heuristic# OpenAI judgeを使う場合
OPENAI_API_KEY=<your-openai-api-key> JUDGE_MODE=auto
```

必ず確認

WANDB_ENTITY は個人名ではなく、W&B Training が有効な Team entity

セキュリティ

APIキーは絶対に共有・コミットしない

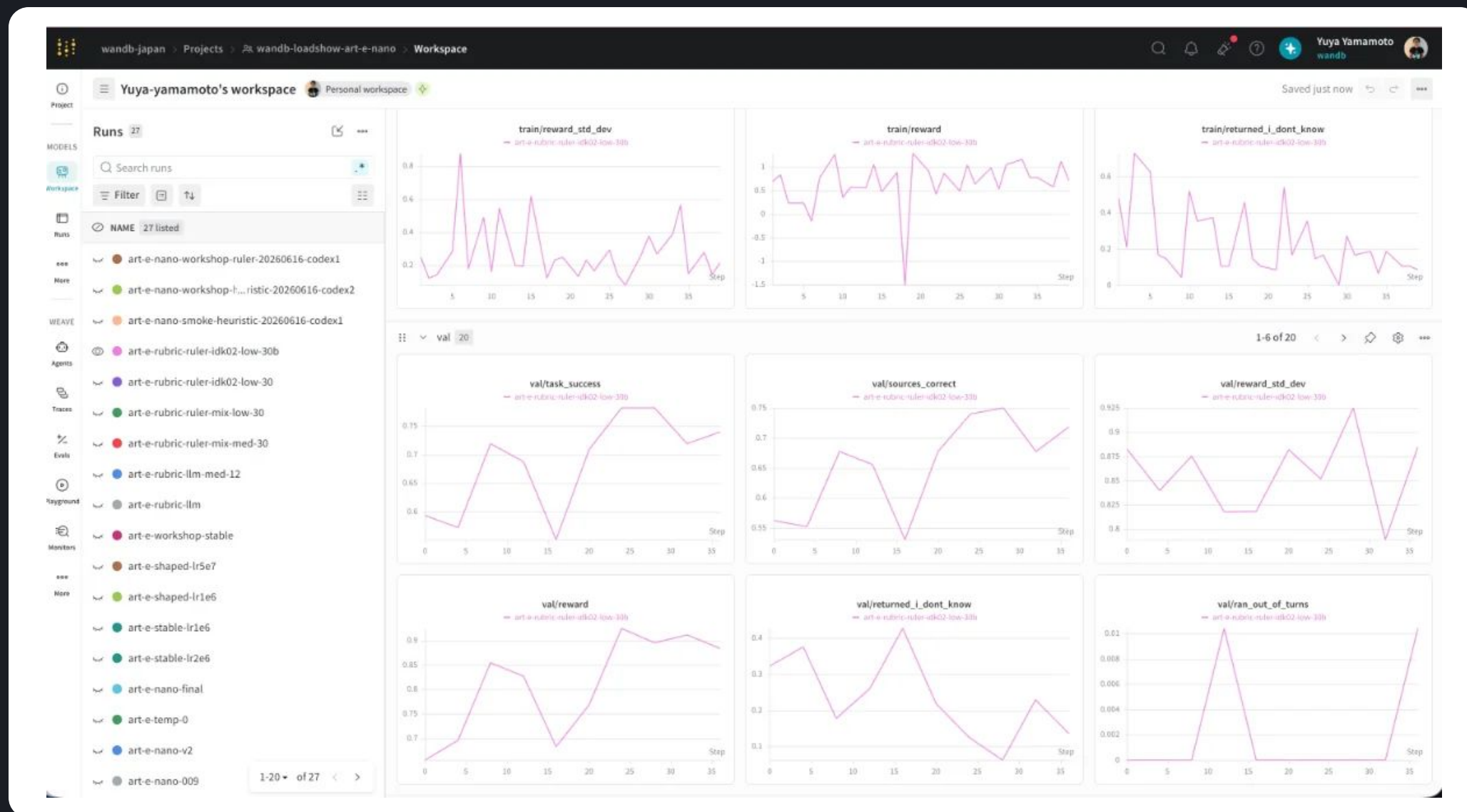
環境を確認してから学習に進む

```
$ uv sync $ uv run python \
  scripts/preflight_check.py
```

PASSすべき項目

- ✓ Python 3.11以上
- ✓ W&B APIキー
- ✓ judge mode
- ✓ W&B Inferenceでの推論
- ✓ .env の読み込み
- ✓ dataset・local DB構築
- ✓ Serverless RLへの登録

学習前のモデルを、固定 validation で評価する



val/answer_correct— 主指標

val/sources_correct— 引用の正しさ

val/task_success— 回答or source 成功

val/returned_i_dont_know— IDK 逃避

val/num_turns— 探索量

Agent loopの中で何が起きているか

- 1 system promptを作る
- 2 tool schemaを提示する
- 3 LLMがJSONでtool callを出す
- 4 Pythonがtoolを実行する
- 5 tool結果を会話履歴に戻す
- 6 return_final_answerで終了する
- 7 FinalRubric から reward を計算する

注目点

`temperature=1.0` で学習rolloutに多様性

validationでは `eval_temperature=0.7`

JSONテキスト方式でtool callを扱う

学習を実行する

```
MODEL_NAME=art-e-nano-$(date +%Y%m%d-%H%M) uv run python -m art_e.train
# ハンズオン②でIDK逃避を観察するため、まず IDK_PENALTY=0 で走らせる例IDK_PENALTY=0 USE_RULER=true JUDGE_MODE=llm
MODEL_NAME=art-e-idk-escape uv run python -m art_e.train
```

ART・E nano ハンズオンの実行フロー



参加者はGitHub repoを利用。zipは確認用の添付です。

step 0評価 → train scenarioをsample → 48 trajectory生成 → reward計算 → GRPOでLoRA更新 → W&B / Weave へ送信

GRPO / GSPO

PPO

clipで更新量を制限し、安全に改善する

現在のRLHF / RLAIIFの標準的な手法

コアとなる考え方

更新前後の方策の確率比を計算し、ある範囲を超えたらクリップする。
Criticが「この行動は平均より良いか」を推定し、行きすぎを防ぎながら改善の余地を残す。

報酬モデルとCriticを別に持つため、**計算・メモリのコストはかかる**

ポイント

- ✓ TRPOをシンプルに近似した手法
- ✓ 実装が簡単で、通常の勾配法で学習できる
- ✓ 安定性と性能のバランスが良い
- ! 報酬モデルとCritic(価値関数)が必要

DPO

選好ペアから 直接学習する

勝ち回答の確率を上げ、負け回答の確率を下げる

コアとなる考え方

同じ入力への「好ましい回答」と「好ましくない回答」のペアを使う。参照モデルと比較しながら、勝ち回答を選ばれやすく、負け回答を選ばれにくくする。報酬モデルやPPO (Critic) を使わず学習できる。

口調・安全性・**単発応答の選好調整** に強い

ポイント

- ✓ 報酬モデルやPPO (Critic) が不要
- ✓ オフラインの選好データだけで学習できる
- ✓ シンプルで実装しやすく、学習も安定しやすい
- ! オンライン探索が弱く、既存データの範囲に寄りやすい

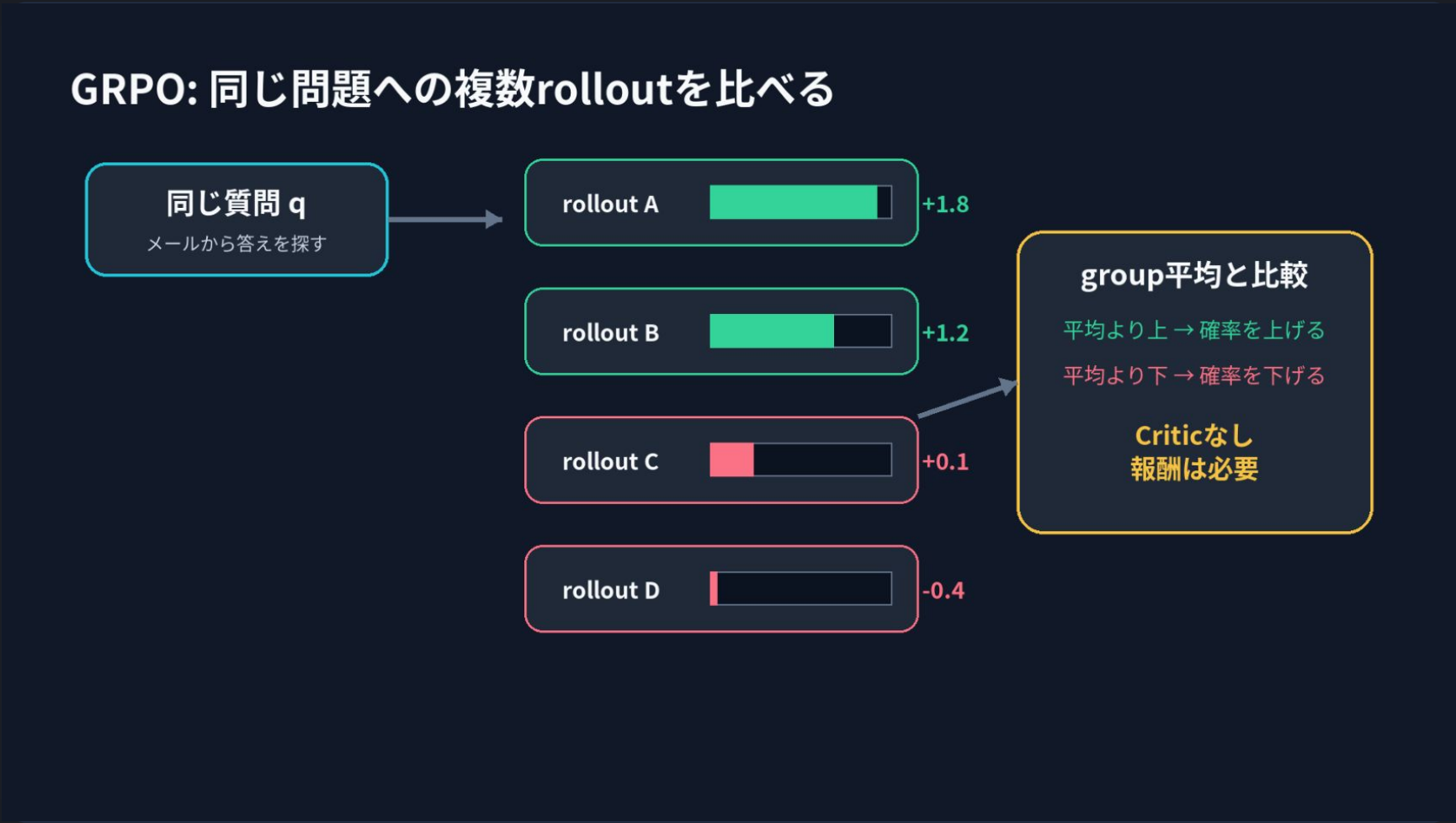
GRPO

同じ問題に複数回答させ、グループ内で
比べる

PPO
Criticが「この行動は平均より良いか」を推定する

GRPO
同じ問題への複数rolloutの報酬を比べ、平均より良い軌跡を強化する

Criticを持たないので、**大きなLLMでもメモリ効率が良い**



GRPOの学習ループ

- 1 同じscenarioを複数回rolloutする
- 2 各trajectoryにrewardを付ける
- 3 グループ平均と標準偏差で相対スコアを作る
- 4 平均より良い軌跡の確率を上げる
- 5 平均より悪い軌跡の確率を下げる
- 6 変えすぎないようにclipやKLで守る

MINI EXAMPLE — 4本のreward





GRPOで起きる典型的な問題

症状	原因	対策
全部同じ報酬	簡単/難しすぎる	難易度調整・rollout数増加・shaping
報酬だけ上がる	報酬ハック	reward breakdownとtrace確認
方策が崩れる	更新が大きすぎる	learning rate低下・clip/KL強化
出力が長くなる	長さが報酬に効く	token/turn penalty・上限設定
IDKに逃げる	誤答ペナルティが強い	IDK penaltyと部分点を調整

GSPO

報酬が回答全体に付くなら、更新もsequence単位にそろえる

GRPO
tokenごとの確率比で更新。長い応答ではノイズが積み上がりやすい

GSPO
回答全体のsequence尤度比で更新。長文・MoE・tool-heavyな軌跡で安定が期待できる

GRPOとGSPO: 更新単位の違い





PPO・DPO・GRPO・GSPOの比較

手法	データ	Critic	更新単位	向く場面
PPO	online rollout	必要	token	報酬モデルがあり探索したい
DPO	offline preference	不要	pair	口調・安全性・単発選好調整
GRPO	online rollout	不要	token	検証可能報酬の推論・tool task
GSPO	online rollout	不要	sequence	長文・MoE・長いAgent軌跡

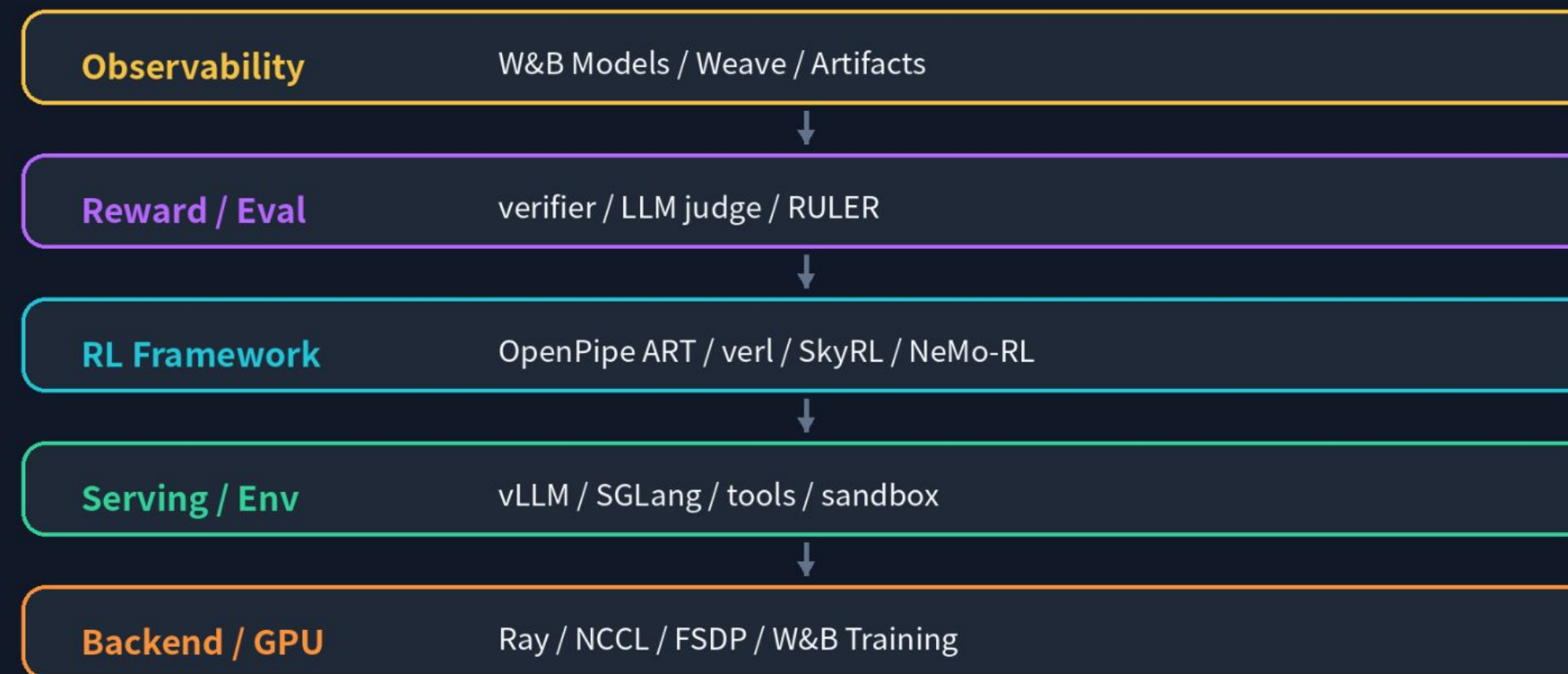
Agentic RLでは、DPOより**online rolloutで環境とやり取りする設計** が中心になる

開発フレームワークと OpenPipe ART

Agentic RL開発スタックの見取り図

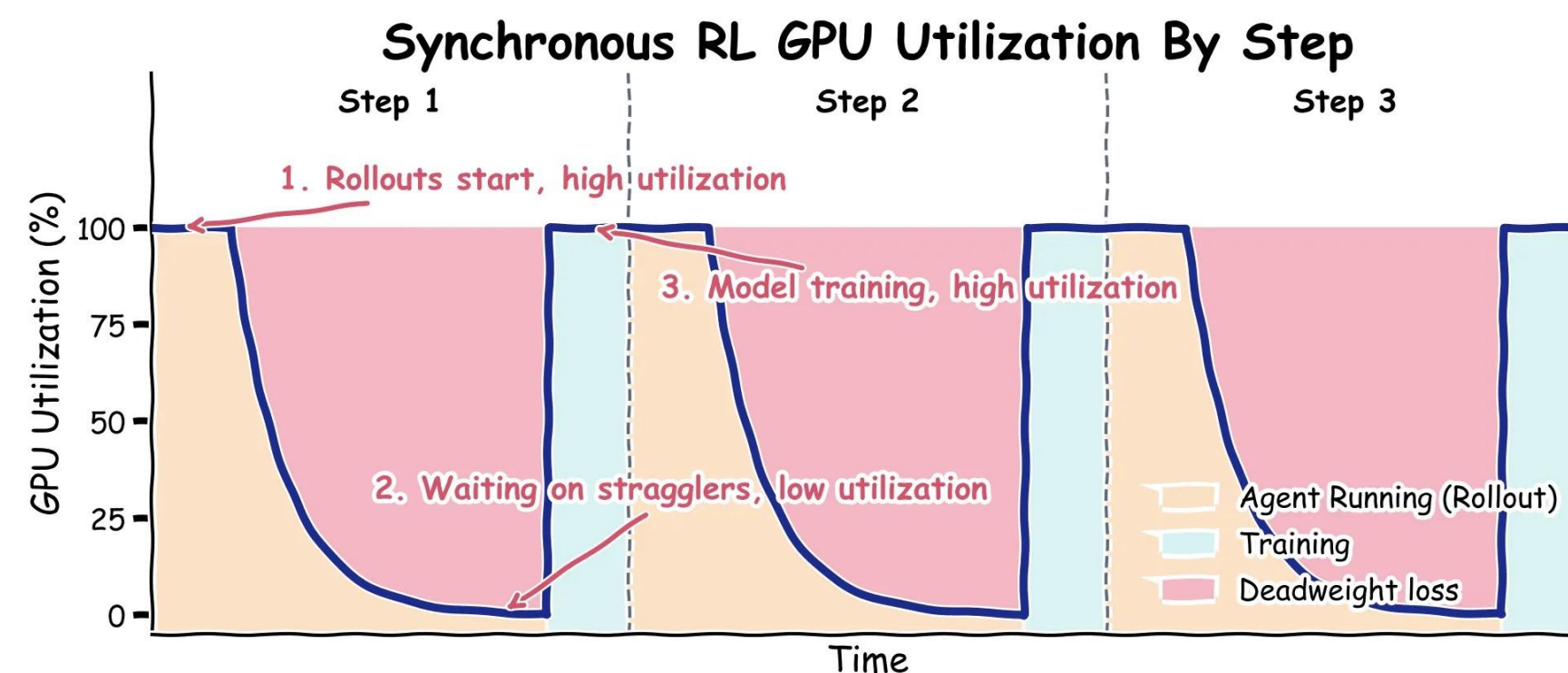
Trainerだけでは足りない

Agentic RL開発スタックの見取り図



Trainerだけでは足りない。rollout、報酬、Serving、観測を1つのシステムとして設計する。

Agentic RLの足かせ: rolloutのstraggler



- rolloutごとにturn数・長さがバラバラ。遅い軌跡を待つ間、GPUが遊ぶ
- 定石: rolloutと学習を非同期化し、大規模分散で押し切る
- だが非同期化は方策ズレ(staleness)を生む——生成時と更新時で方策が違う
- LLMでは系列の対数尤度の積で効くため、わずかなズレが系列長に応じて増幅。単純な非同期化は簡単ではない

WHY ART・理論

重要度サンプリング：古い方策のデータを再利用する

- 非同期化すると、生成時の方策と学習時の方策がずれる (staleness)
- 古い方策のサンプルを、新しい方策での期待値に「換算」して使いたい
- その重みが、確率比 $r = \pi_{\text{new}} / \pi_{\text{old}}$
- PPO / GRPO はこの比を **clip** して安全に使う

重要度比

$$r(a) = \pi_{\text{new}}(a) / \pi_{\text{old}}(a)$$

補正された目的 (期待アドバンテージ)

$$E[r(a) \cdot A(a)]_{a \sim \pi_{\text{old}}}$$

$A(a)$ = アドバンテージ (その行動の相対的な良さ)

trajectory単位でそのままやると崩壊する

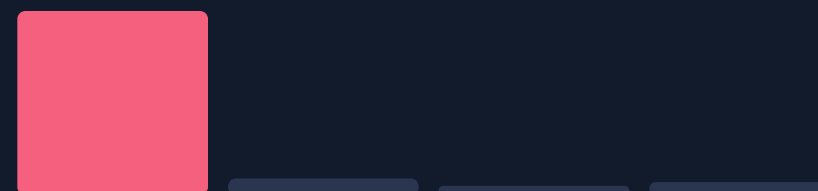
軌跡 $\tau = (a_1, a_2, \dots, a_T)$ $T =$ 応答トークン数(数百~数千)

$$r(\tau) = \prod_{t=1..T} \pi_{\text{new}}(a_t) / \pi_{\text{old}}(a_t)$$

$$= \exp\left(\sum_{t=1..T} (\log \pi_{\text{new}} - \log \pi_{\text{old}})\right)$$

各トークンの小さなずれがT回かけ算で累積する

- 比が天文学的に小さく／大きくなり、
- 有効サンプルサイズが崩壊——一部のサンプルだけが巨大な重み、残りは ≈ 0

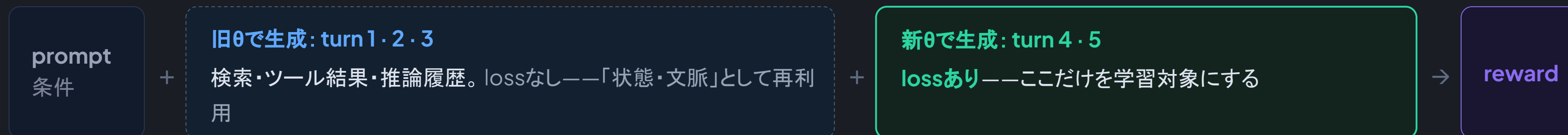


1つのサンプルに重みが集中し、学習が不安定に

AlphaGoは1手=1行動でずれは1ステップぶん。LLMは1応答=数百トークンないしそれ以上 なので、軌跡単位の補正はそのままでは壊れやすい

partial rollout: 古い部分は「文脈」、新しい部分だけ学習

MoonshotのKimiなどが採用。長い軌跡を途中保存し、更新後の重みで続きを生成する



利点

prefix生成は無駄にならない。探索履歴・ツール結果が次の行動の状態として効く

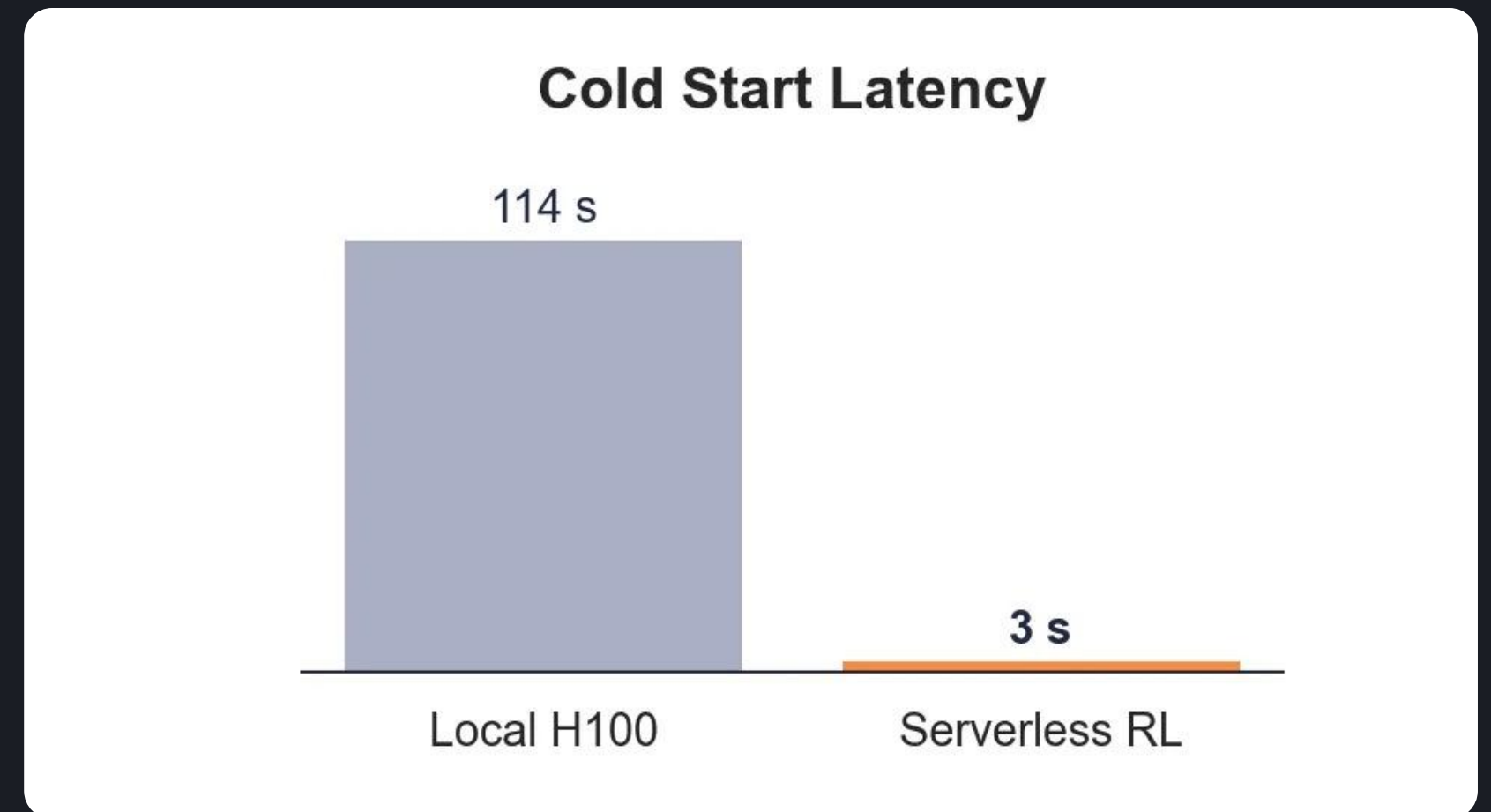
難しさ

旧turnへのcredit assignmentは弱く、staleness管理やmask設計が複雑になる

WHY ART

別解: 非同期化ではなく Serverless化

- stalenessを増やさず、Serverless化でGPU利用率に別軸から対処
- アイドルGPUを基盤側がmanagedに吸収・共有。コールドスタートも短い
- 既存のPython Agentを低摩擦でRL化。節約するのは **GPU時間だけでなく人間の時間**

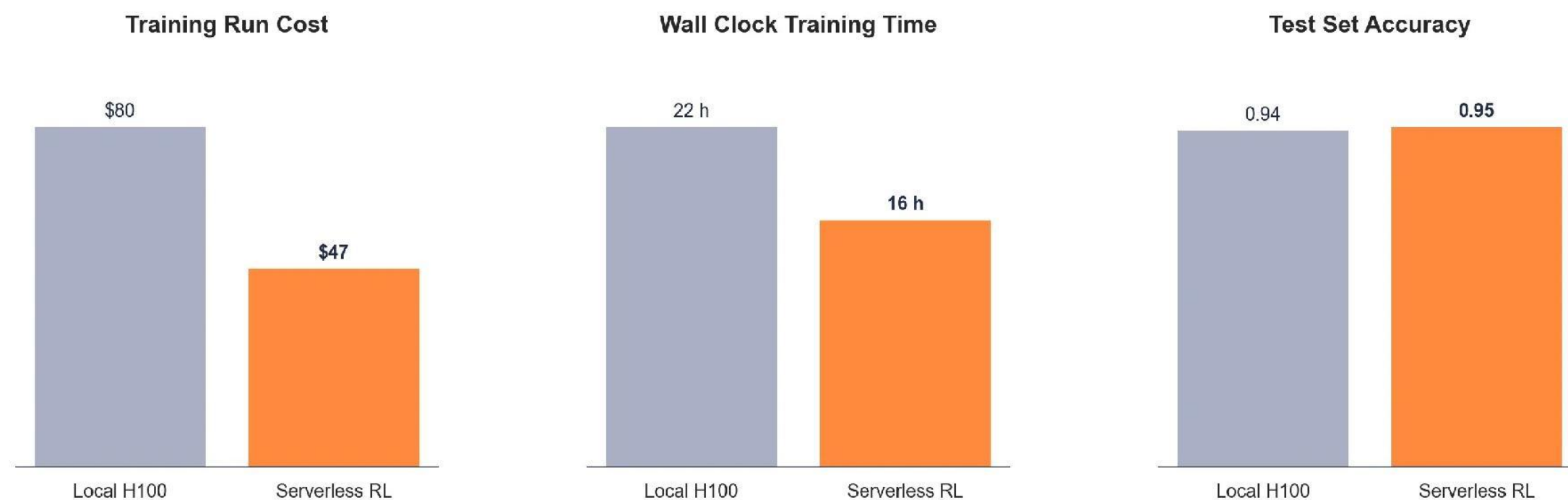


コールドスタート: Local H100 114秒 → Serverless RL 3秒

Serverlessの効果: 安く・速く・精度はそのまま

Local H100との比較(同一の学習設定)

Serverless RL vs. Local H100 Training



コスト **\$80 → \$47**、時間 **22h → 16h**、精度は **0.94 → 0.95** と同等

どのフレームワークを選ぶか

最高スループットを追う

verl・SkyRLなど分散非同期基盤を検討

既存 Agent を素早く RL 化する

ART を検討

ローカル GPU で小さく試す

ART LocalBackend

GPU 運用を避けて試す

W&B Training ServerlessBackend

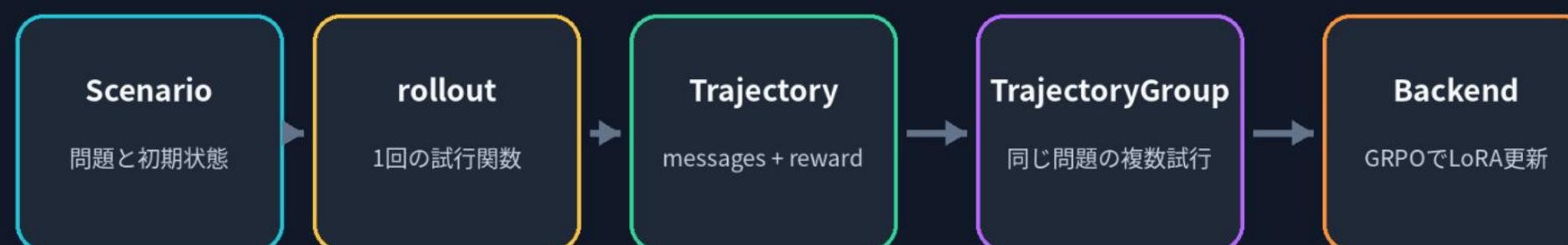
監査・系譜・traceを残す

W&B Models + Artifacts + Weave

既存のPython Agentを、RLで学習可能な形にする OSS

ARTの基本単位

OpenPipe ARTの基本単位



既存のPython Agentを学習対象のtrajectoryに変換する

W&B の基本 — run を記録する4つの呼び出し

```
import wandb

run = wandb.init(
    entity=ENTITY, project=PROJECT,
    config={"lr": 5e-6, "base_model": BASE_MODEL}) # ① run を開始

wandb.log({"reward": 0.62, "loss": 0.41}, step=step) # ② 時系列

artifact = wandb.Artifact("retail-lora", type="model")
artifact.add_dir(checkpoint_path)
wandb.log_artifact(artifact) # ③ 成果物を記録

wandb.finish() # ④ run を閉じる
```

呼び出し	役割
① <code>init</code>	run を開始。entity / project / config を固定する
② <code>log</code>	reward・lossなどをstep付きで時系列に記録
③ <code>log_artifact</code>	checkpoint・datasetを版管理つき成果物として保存
④ <code>finish</code>	run を閉じ、サマリと系譜を確定する

ART では `model.log(...)` がこの `init` → `log` を内部で担う。checkpoint の `log_artifact` と Registry 昇格は**教材側で明示的に** 書く。

Weave の基本 — @weave.op() で実行を追う

```
import weave

weave.init(f"{ENTITY}/{PROJECT}")      # W&B と同じ project に紐づく

@weave.op()                             # 入出力・時間・子呼び出しを trace に
async def rollout(model_name, scenario_id):
    completion = await client.chat.completions.create(...)
    reward = await score_reward(...)    # 入れ子の op が木になる
    return {"reward": reward}

# OpenAI / LLM 呼び出しは自動でトレースされる
```

要素	役割
<code>weave.init</code>	W&B と同じ project に trace を保存
<code>@weave.op()</code>	関数の入出力・所要時間・例外を記録
入れ子	op の中で op を呼ぶと 実行木 になる
自動トレース	OpenAI 等の LLM 呼び出しは自動で残る

W&B が**時系列と系譜**、Weave が**各実行の木**。rollout ・ call_tool ・ score_reward ・ evaluate に @weave.op() を付けて、なぜ成功 / 失敗したかを辿れるようにする。

ARTの学習ループ

```
model = art.TrainableModel(  
    name="art-e-nano-workshop",  
    project="art-e-nano",  
    base_model="OpenPipe/Qwen3-14B-Instruct",  
)  
await model.register(backend)  
  
groups = await art.gather_trajectory_groups(  
    art.TrajectoryGroup(  
        rollout(model, scenario) for _ in range(4)  
    ) for scenario in batch  
)  
  
result = await backend.train(model, groups, learning_rate=1.2e-5)  
await model.log(groups, metrics=result.metrics, step=result.step)
```

register

backendにモデルを登録

gather

複数rolloutで軌跡を集める

train

GRPOでLoRAを更新

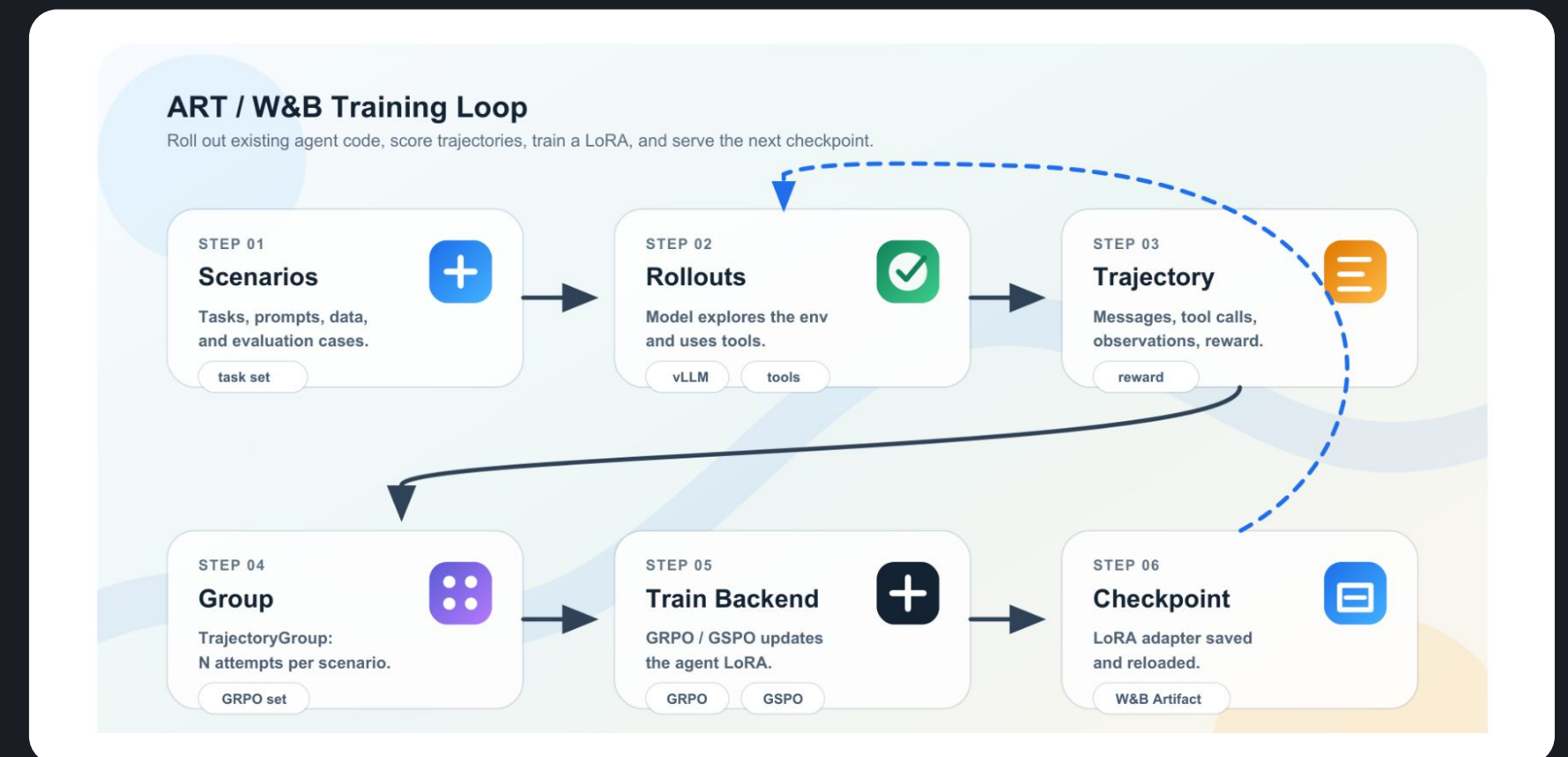
model.log

各stepの結果(軌跡・報酬・指標)を残す

W&B TRAINING

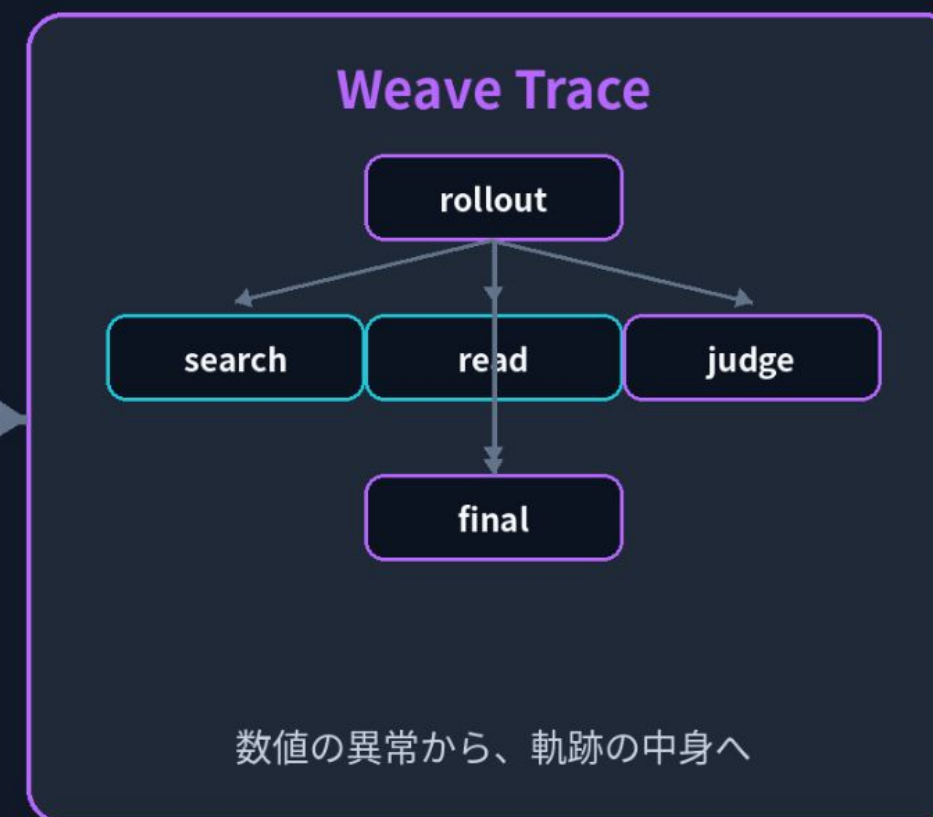
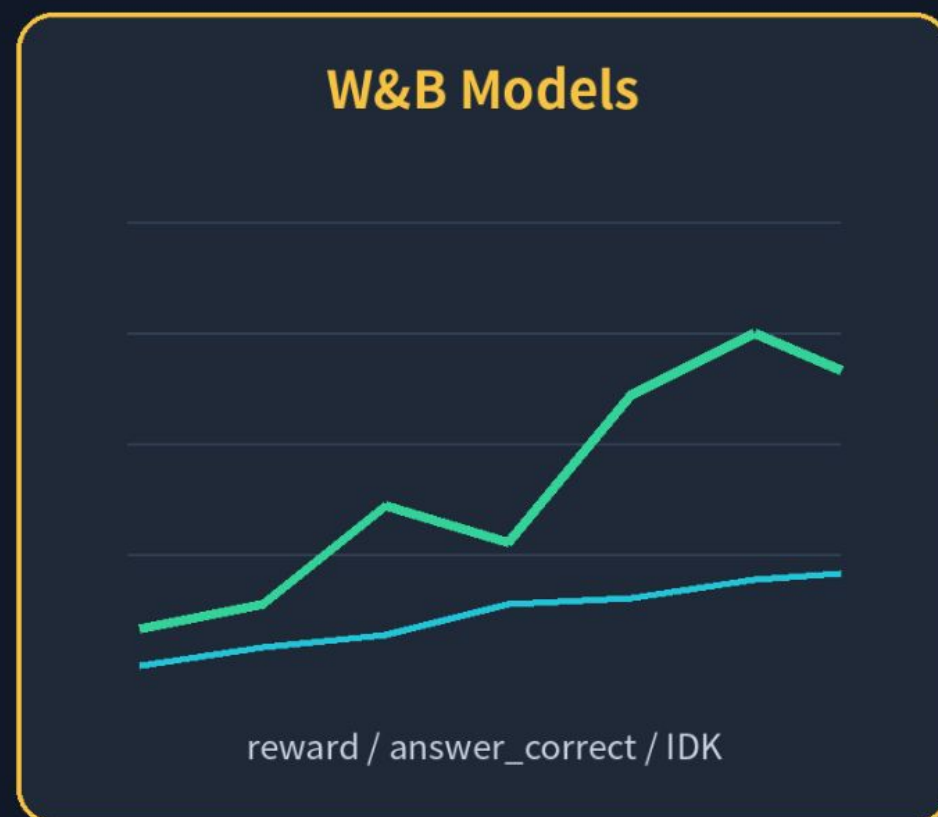
ARTを入口に、Serverless RLでGPU運用を隠す

- Serverless RLはW&B Trainingの一部
- ARTのServerlessBackendから使う
- training infrastructureをmanagedに扱う
- LoRA・metrics・Artifacts・Weaveと連携
- 参加者がGPUを用意しなくても実行できる



W&Bで全体の傾向、Weaveで1回の中身を見る

W&Bは曲線、Weaveは実行木



ハンズオン② 結果を見て改善する

W&Bで学習曲線を見る

<code>train/reward</code>	報酬平均。上がっても成功とは限らない
<code>train/reward_std_dev</code>	グループ内に学習信号があるか
<code>val/answer_correct</code>	held-out主指標
<code>val/sources_correct</code>	検索行動の改善
<code>val/returned_i_dont_know</code>	IDK逃避・崩壊の兆候
<code>val/num_turns</code>	速度と探索量のproxy

学習が進むと、IDKに逃げ始める

- エージェントが「分かりません(I don't know)」を多用し始める
- このハンズ `IDK_PENALTY=0.2` を、逃避再現 `0` にして実行。誤答は大オンでは既のため明示的きな減点、IDKは無傷定のに
- 結果、「自信がなければ答えない」方が期待報酬が高くなり、安全側へ収束

```
# 逃避を再現(IDKペナルティ0)
IDK_PENALTY=0 USE_RULER=true JUDGE_MODE=llm
MODEL_NAME=art-e-idk-escape uv run python -m art_e.train
```

症状(overall rewardでは見えにくい)

returned_i_dont_know



answer_correct



報酬を項目ごとにログし、良くない勾配の向きを直す

- 1 overall rewardだけでなく、報酬の 各項目を個別に W&Bへログ
`answer_correct / sources_correct / returned_i_dont_know / num_turns / format`
- 2 どの項目で稼ぎ、どこが犠牲かを切り分け、失敗モードを特定
- 3 処方: `IDK_PENALTY 0 → 0.2`。IDKの期待値を下げ、探索方向へ勾配を向ける

```
IDK_PENALTY=0.2 USE_RULER=true JUDGE_MODE=llm
MODEL_NAME=art-e-idk-fixed uv run python -m art_e.train
```

IDK_PENALTY=0 (前)

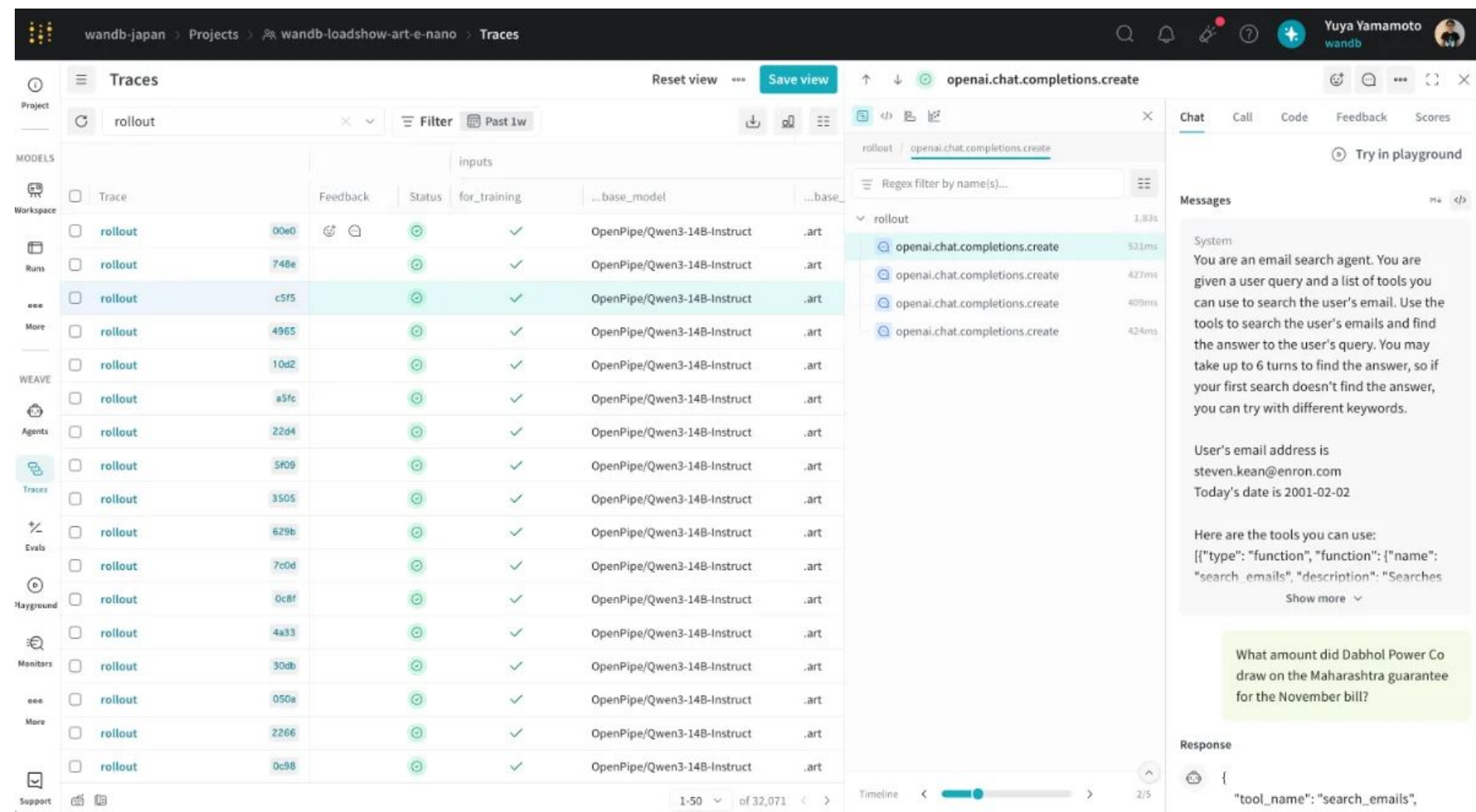
IDKに逃避。answer_correct 横ばい



IDK_PENALTY=0.2 (後)

returned_i_dont_know ↓ / answer_correct ↑

数値の裏にある行動を見る



The screenshot displays the Weave interface for monitoring and debugging AI workflows. The main panel shows a list of traces for a project named 'wandb-japan'. The traces are filtered by 'rollout' and sorted by 'message_id'. The selected trace is 'rollout' with message_id 'c5f5'. The detailed view on the right shows the trace's execution path, including the 'openai.chat.completions.create' step. The chat interface on the right shows the system prompt and the user's query, along with the model's response.

Trace	message_id	Status	for_training	...base_model	...base...
rollout	00e0	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	748e	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	c5f5	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	4965	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	10d2	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	a5fc	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	22d4	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	5f09	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	3505	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	629b	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	7c0d	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	0c8f	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	4a33	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	30db	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	050a	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	2266	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art
rollout	0c98	Success	✓	OpenPipe/Qwen3-14B-Instruct	.art

Chat interface details:

- System: You are an email search agent. You are given a user query and a list of tools you can use to search the user's email. Use the tools to search the user's emails and find the answer to the user's query. You may take up to 6 turns to find the answer, so if your first search doesn't find the answer, you can try with different keywords.
- User's email address is: steven.kean@enron.com
- Today's date is 2001-02-02
- Here are the tools you can use: [{"type": "function", "function": {"name": "search_emails", "description": "Searches for emails in the user's inbox."}}]
- User's query: What amount did Dabhol Power Co draw on the Maharashtra guarantee for the November bill?
- Response: {"tool_name": "search_emails", ...}

- ✓ 最初の検索クエリは妥当か
- ✓ 正しい message_id が出ているか
- ✓ そのメールを読んでいるか
- ✓ source を正しく返しているか
- ! IDKへ逃げていないか
- ! 同じtool callを繰り返していないか
- ✗ JSONが壊れていないか

報酬を少し変えてみる

IDKへの逃避を少し強く抑える

```
IDK_PENALTY=0.4 uv run python -m art_e.train
```

簡易報酬にする

```
SIMPLE_REWARD=true uv run python -m art_e.train
```

RULERを補助加点として混ぜる

```
USE_RULER=true RULER_WEIGHT=0.2 uv run ...
```

比較すること

answer_correct は上がるか

sources_correct は上がるか

returned_i_dont_know は下がるか

trace の品質は悪化していないか

train rewardの最大値では選ばない



answer・sources・IDK・turn数と比較してから採用する

トラブルシューティング



RL の成功率が崩れるときの診断

症状	何が起きているか	対応
policy update が不安定	1回の更新で方策が動きすぎ、次の rollout 分布が急変	lr を下げる・KL penalty・clip 強化・崩壊前へ fork
rollout-training mismatch	推論 engine と学習 backend で logprob がずれ、実質 off-policy 化	train/inference を揃える・truncated importance sampling
reward over-optimization	proxy reward は上がるが真の success / 品質が下がる	verifier 主軸・soft reward の重み↓・early stopping・gate
sparse で信号が消える	group 全員が同報酬で advantage が立たない	rollouts/scenario↑・balanced shaping・curriculum 調整
dense が向きを誤らせる	multi-turn で部分点行動へ寄り、task success が伸びない	outcome / state-action を anchor に・補助は減点中心
長文化・反復・打ち切り	entropy collapse、overlong 出力で報酬がノイズ化	max tokens/turns 設定・truncation penalty・clip/KL 調整

成功率低下は「失敗」ではなく観測と設計を改善する**シグナル**。採用判断は train metric ではなく fresh held-out eval と Weave trace で行う。— TRPO/PPO・DeepSeekMath・DAPO・Iterative Reward Calibration

単調改善を仮定しない

RL の成功率が崩れるときの診断

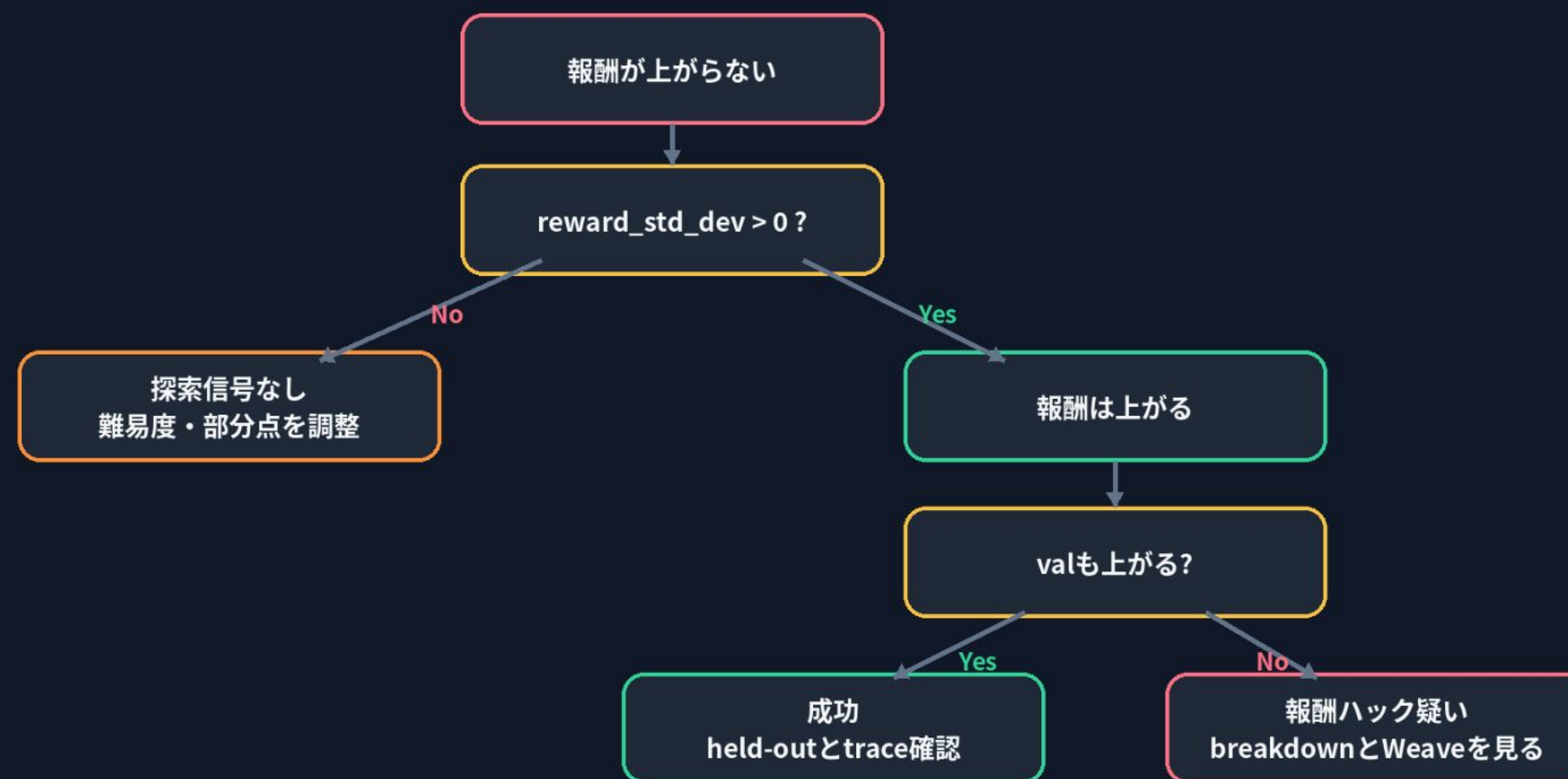
症状	何が起きているか	対応
policy update が不安定	1回の更新で方策が動きすぎ、次の rollout 分布が急変	lr を下げる・KL penalty・clip 強化・崩壊前へ fork
rollout-training mismatch	推論 engine と学習 backend で logprob がずれ、実質 off-policy 化	train/inference を揃える・truncated importance sampling
reward over-optimization	proxy reward は上がるが真の success / 品質が下がる	verifier 主軸・soft reward の重み↓・early stopping・gate
sparse で信号が消える	group 全員が同報酬で advantage が立たない	rollouts/scenario↑・balanced shaping・curriculum 調整
dense が向きを誤らせる	multi-turn で部分点行動へ寄り、task success が伸びない	outcome / state-action を anchor に・補助は減点中心
長文化・反復・打ち切り	entropy collapse、overlong 出力で報酬がノイズ化	max tokens/turns 設定・truncation penalty・clip/KL 調整

成功率低下は「失敗」ではなく観測と設計を改善する**シグナル**。採用判断は train metric ではなく fresh held-out eval と Weave trace で行う。 — TRPO/PPO · DeepSeekMath · DAPO · Iterative Reward Calibration

報酬が上がらない

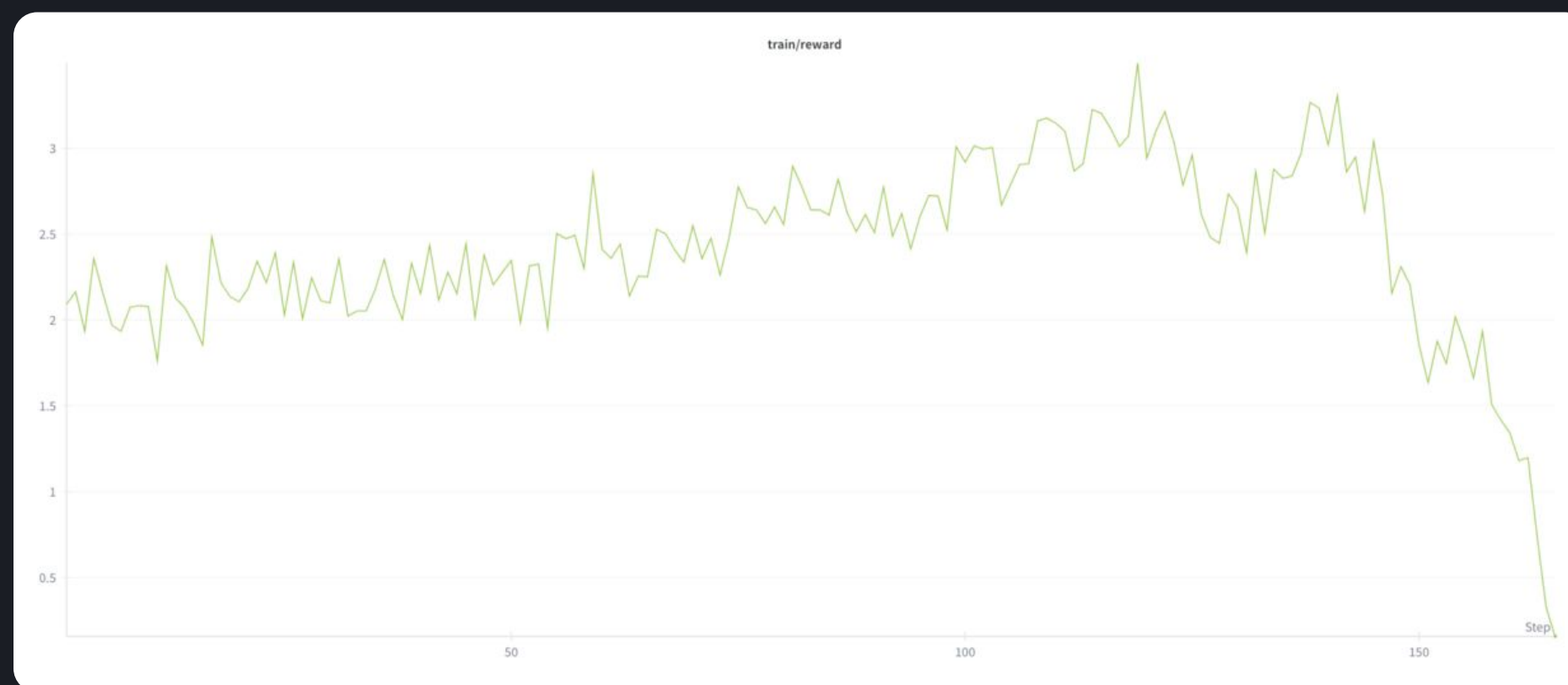
まず「学習信号が存在するか」を見る

学習がうまくいかないときの切り分け



崩壊時は learning rate / clip / KL / max turns を確認

rewardが急落し、formatやtool callが壊れる



よくある原因

learning rateが高い / clip・KLが弱い
rewardの一部が強すぎる
max_tokensやturn上限が合っていない

対策

学習率を下げる / 良いcheckpointへ戻す
RULER・soft rewardの重みを下げる
別seedで再現するか確認する

報酬は上がるが、能力が伸びない

最も危険なケース

症状

`train/reward` ↑上がる

`val/answer_correct` →横ばい

tool callだけ増える・長い説明だけ増える・judgeに好まれる表現に寄る・sourceが悪化

対策

rewardをbreakdownして見る

成功率・source率・turn数を分けて見る

negative例を増やす

held-out evalで比較する

Weaveで前後のtraceを見比べる

伸びが頭打ち

方向性は合っているが目標に届かない

確認すること

- 元モデルの能力上限ではないか
- train scenarioの質・量・多様性は十分か
- validationがtrainとズレすぎているか
- ideal trajectoryをSFTで見せたか

対策レバー

データを増やす / scenarioを多様化する
SFT warm start を追加する
reward contract を見直す
GRPOからGSPOも検討する

モデルは仕様の穴を見つける

報酬ハック: モデルは仕様の穴を見つける



tool call稼ぎ
上限とcost penalty

形式だけ遵守
outcome主軸

IDK逃避
IDK penalty調整

長文化
token/turnコスト

source水増し
source数制限

学習時に見るべきメトリクス一覧

主成果	<code>answer_correct, task_success</code>	検索行動	<code>ever_found / ever_read / sources_correct</code>
安全性	<code>returned_i_dont_know, wrong_answer</code>	形式	<code>cant_parse_tool_call, bad_tool_call_*</code>
効率	<code>num_turns, prompt / completion_tokens</code>	学習信号	<code>reward, reward_std_dev, zero_variance</code>
コスト		<code>time, API usage, judge cost</code>	

Agentの行動面を狭くする

学習環境でも、本番環境でも

- 書き込み系toolはsandboxで扱う
- tool引数のschema validationを厳しく
- traceの機密情報はredactionする
- 不可逆な操作は人間承認を挟む
- permission scopeを最小化する
- 採用前にheld-outとnegative testを通す

まとめと次の一歩



自社Agentへ展開する最小ロードマップ

週	やること	成果物
1	狭いタスクを選ぶ	reward contract
2	環境とtoolを固定する	deterministic eval, trace logging
3	baselineと失敗例を集める	W&B report
4	SFT warm startを試す	初期policy
5	GRPOで小さく学習	reward curve, trace comparison
6	held-out gateと運用設計	checkpoint lifecycle

報酬関数を書く前に、仕様を書く

Reward Contract ワーク

Reward Contractを書く

成功条件	正しい答え + 正しいsource
許容失敗	見つからない場合のI don't know
危険な失敗	誤答の断言、誤source
効率条件	少ないturn、無駄なreadを避ける
形式条件	parse可能なtool callとfinal
観測項目	answer_correct, sources_correct, num_turns

報酬関数を書く前に、仕様として合意する

あなたの業務を Agentic RL化するなら

01

任せたい狭いタスクは？

02

観測は何か？

03

行動は何か？

04

最終成功はどう判定？

05

危険な失敗は何か？

06

どのtraceを必ず見る？

出力

1文のタスク定義

tool一覧

reward contractの初版

Agentic RLは「仕事の進め方」を学習する

- 従来型LLM RLは単発応答の品質最適化に近い
- Agentic RLは観測・tool・行動・報酬を含むtrajectory最適化
- GRPOは複数rolloutを相対比較し、Criticなしで学習する
- GSPOはsequence単位で報酬と更新の単位をそろえる
- ARTは既存Python Agentを低摩擦でRL化する
- W&BとWeaveは報酬曲線と実行traceをつなぐ
- 成功の鍵は、環境・報酬・観測・デバッグ

ART-E nano 実行コマンド集

```
# 事前確認 uv run python scripts/preflight_check.py # 学習 MODEL_NAME=art-e-nano-$(date +%Y%m%d-%H%M) uv run python -m
art_e.train # プロンプトのみモデルとの比較 uv run python -m art_e.evaluate.benchmark_prompted_models # よく使う
override JUDGE_MODE=heuristic uv run python -m art_e.train IDK_PENALTY=0 USE_RULER=true JUDGE_MODE=llm
MODEL_NAME=art-e-idk-escape uv run python -m art_e.train IDK_PENALTY=0.2 USE_RULER=true JUDGE_MODE=llm
MODEL_NAME=art-e-idk-fixed uv run python -m art_e.train SIMPLE_REWARD=true uv run python -m art_e.train USE_RULER=true
RULER_WEIGHT=0.2 uv run python -m art_e.train
```



用語集

Policy	観測から行動を選ぶモデル
Trajectory	会話・tool・観測・報酬の履歴
Reward	軌跡の良さを表す点数
LLM judge	LLMで品質や正誤を評価する仕組み
GSPO	sequence単位の確率比で更新

Rollout	1回の試行
Scenario	学習・評価する1つの問題設定
Verifier	正しさを機械的に判定する仕組み
GRPO	グループ相対比較でCriticなしに学習
RULER	ARTのLLM judge系reward支援

参考資料

- OpenPipe blog — ART-E: How We Built an Email Research Agent That Beats o3
- OpenPipe ART Docs / GitHub
- W&B Serverless Training Docs
- W&B White Paper — Agentic Reinforcement Learningの基礎と実践
- 添付コード — art-e-nano-main